

2/25/2025

Assignment-02

Course Title: Data Structure and Algorithms-II Lab
Course Code: CSE 2202

Prepared by:

Md. Ehasan Mahmud

Roll: 202122104026

Department: Computer Science and Engineering

Submitted to:

Dr. Sujan Chandra Roy

Lecturer

**Department of Computer Science and
Engineering**

Kishoreganj University.

1.AVL tree implementation (inserting numbers from 1 to 8):

- **Perform LL on the root of a tree**
- **Perform RR on the root of a tree**
- **Perform LR on the root of a tree**
- **Perform RL on the root of a tree**
- **Remove the value of an element from the tree. Show the tree before and after rebalancing.**

Code:

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
struct Node
```

```
{
    int key;
    Node *left;
    Node *right;
    int height;
};
```

```
int height(Node *n)
```

```
{
    return n ? n->height:0;
}
```

```
int max(int a,int b)
```

```
{
    return (a>b) ? a:b;
}
```

```
Node* newNode(int key)
```

```
{
    Node* node=new Node();
    node->key=key;
    node->left=NULL;
    node->right=NULL;
    node->height=1;
    return node;
}
```

```

Node* rightRotate(Node *y)
{
    Node *x=y->left;
    Node *T2=x->right;
    x->right=y;
    y->left=T2;
    y->height=max(height(y->left),height(y->right))+1;
    x->height=max(height(x->left),height(x->right))+1;
    return x;
}

Node* leftRotate(Node *x)
{
    Node *y=x->right;
    Node *T2=y->left;
    y->left=x;
    x->right=T2;
    x->height=max(height(x->left),height(x->right))+1;
    y->height= max(height(y->left),height(y->right))+1;
    return y;
}

int getBalance(Node *n)
{
    return n ? height(n->left)-height(n->right):0;
}

Node* insert(Node* node,int key)
{
    if (!node)
        return newNode(key);
    if (key<node->key)
        node->left=insert(node->left,key);
    else if (key>node->key)
        node->right=insert(node->right,key);
    else
        return node;
}

```

```

node->height=1+max(height(node->left),height(node->right));
int balance=getBalance(node);
if (balance>1 && key<node->left->key)
    return rightRotate(node);
if (balance<-1 && key>node->right->key)
    return leftRotate(node);
if (balance>1 && key>node->left->key)
{
    node->left=leftRotate(node->left);
    return rightRotate(node);
}
if (balance<-1 && key<node->right->key)
{
    node->right=rightRotate(node->right);
    return leftRotate(node);
}
return node;
}

void preOrder(Node *root)
{
    if (root!=NULL)
    {
        cout<<root->key<<" ";
        preOrder(root->left);
        preOrder(root->right);
    }
}

Node* minValueNode(Node* node)
{
    Node* current=node;
    while(current->left)
        current=current->left;
    return current;
}

```

```

Node* deleteNode(Node* root,int key)
{
    if (!root)
        return root;
    if (key<root->key)
        root->left=deleteNode(root->left,key);
    else if (key>root->key)
        root->right=deleteNode(root->right,key);
    else
    {
        if (!root->left || !root->right)
        {
            Node* temp=root->left ? root->left:root->right;
            if (!temp)
            {
                temp=root;
                root=NULL;
            }
            else
                *root=*temp;

            delete temp;
        }
        else
        {
            Node* temp=minValueNode(root->right);
            root->key=temp->key;
            root->right=deleteNode(root->right,temp->key);
        }
    }
    if (!root)
        return root;
    root->height=1+max(height(root->left),height(root->right));
    int balance=getBalance(root);
    if (balance>1 && getBalance(root->left)>=0)

```

```

        return rightRotate(root);
    if (balance>1 && getBalance(root->left)<0)
    {
        root->left=leftRotate(root->left);
        return rightRotate(root);
    }
    if (balance<-1 && getBalance(root->right)<=0)
        return leftRotate(root);
    if (balance<-1 && getBalance(root->right)>0)
    {
        root->right=rightRotate(root->right);
        return leftRotate(root);
    }
    return root;
}

int main()
{
    Node *root=NULL;
    for (int i=1; i<=8; ++i)
        root=insert(root,i);
    cout << "AVL tree:"<<endl;
    preOrder(root);
    cout << endl;
    Node *rootLL = rightRotate(root);
    cout<<endl;
    cout << "After LL rotation on root:"<<endl;
    preOrder(rootLL);
    cout<<endl;
    root=NULL;
    for (int i=1; i<=8; ++i)
        root=insert(root,i);
    Node *rootRR=leftRotate(root);
    cout<<endl;
    cout<<"After RR rotation on root:"<<endl;

```

```

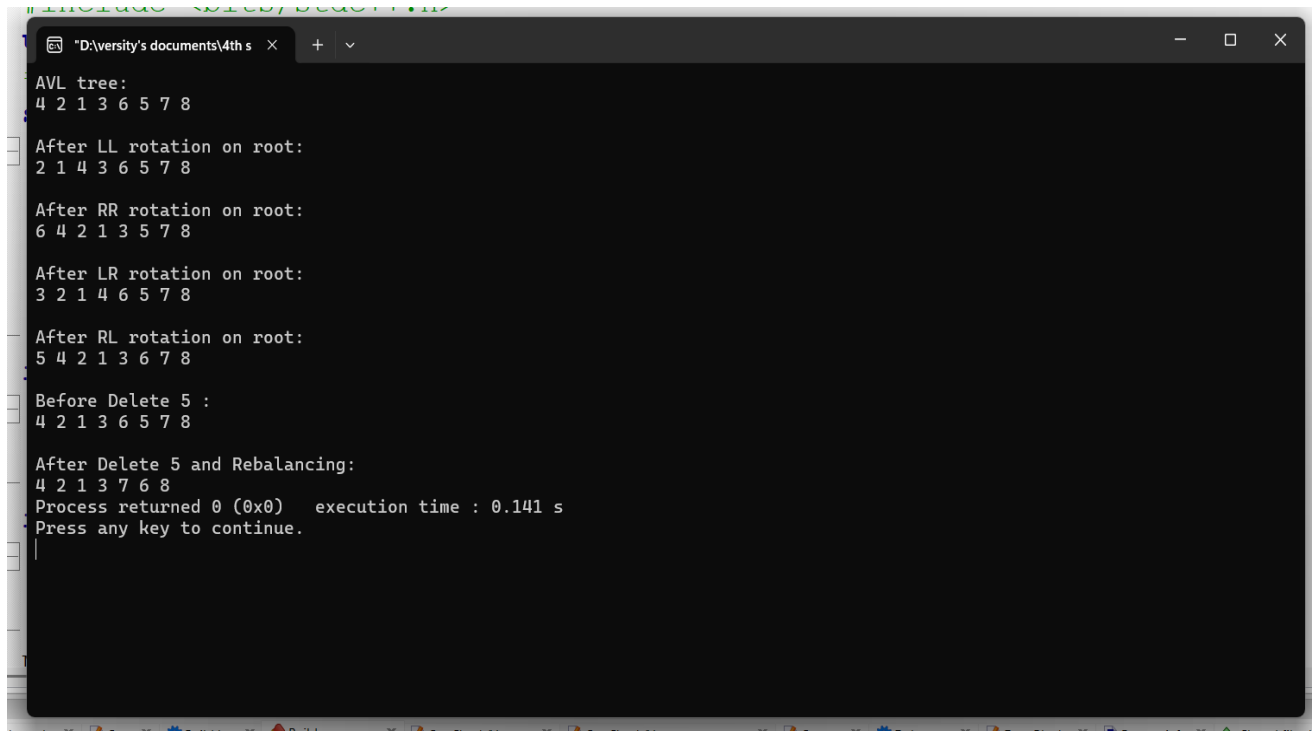
preOrder(rootRR);
cout<<endl;

root=NULL;
for (int i=1; i<=8; ++i)
    root=insert(root,i);
root->left=leftRotate(root->left);
Node *rootLR=rightRotate(root);
cout<<endl;
cout << "After LR rotation on root:"<<endl;
preOrder(rootLR);
cout<<endl;
root=NULL;
for (int i=1; i<=8; ++i)
    root=insert(root,i);
root->right=rightRotate(root->right);
Node *rootRL=leftRotate(root);
cout<<endl;
cout<<"After RL rotation on root:"<<endl;
preOrder(rootRL);
cout<<endl;
root=NULL;
for (int i=1; i<=8; ++i)
    root=insert(root,i);
cout<<endl;
cout<<"Before Delete 5 :"<<endl;
preOrder(root);
cout<<endl;
root=deleteNode(root,5);
cout<<endl;
cout<<"After Delete 5 and Rebalancing:"<<endl;
preOrder(root);
return 0;

```

}

Output:

A screenshot of a terminal window with a dark background and light-colored text. The window title bar shows the file path "D:\iversity's documents\4th s" and standard window controls. The terminal output displays the following text:

```
AVL tree:
4 2 1 3 6 5 7 8

After LL rotation on root:
2 1 4 3 6 5 7 8

After RR rotation on root:
6 4 2 1 3 5 7 8

After LR rotation on root:
3 2 1 4 6 5 7 8

After RL rotation on root:
5 4 2 1 3 6 7 8

Before Delete 5 :
4 2 1 3 6 5 7 8

After Delete 5 and Rebalancing:
4 2 1 3 7 6 8
Process returned 0 (0x0)   execution time : 0.141 s
Press any key to continue.
```

2. Write a program that will implement and analyze the effectiveness of skip lists, a probabilistic data structure that can efficiently perform search, insert, and delete operations.

Solution:

Effectiveness of skip list: Skip lists are a data structure that offers efficient search, insertion, and deletion in a sorted sequence. It's a probabilistic alternative to balanced trees (like AVL or Red-Black Trees), providing $O(\log n)$ time complexity on average for these operations.

By using multiple levels of linked lists, skip lists allow for fast searching by skipping over portions of data, which reduces the search space quickly. New elements are inserted at a randomly chosen level, ensuring the list stays balanced without requiring heavy rebalancing like in AVL trees.

Benefits:

Faster
Easier
Logarithmic

Code:

```
#include <bits/stdc++.h>

using namespace std;

template <typename T>

class FastLane{

private:

    struct Element{

        T data;

        vector<Element*> next;

        Element(T val,int level):data(val),next(level+1,NULL){}

    };

    const int MAX_HEIGHT;

    const float CHANCE;

    int current_height;

    Element* head;

    mt19937 rng;

    uniform_real_distribution<float> random_gen;

public:

    FastLane(int max_height=16,float
chance=0.5):MAX_HEIGHT(max_height),CHANCE(chance),current_height(0),rng(std::random_device{}()),random_gen(0
.0,1.0){

        head=new Element(std::numeric_limits<T>::min(),MAX_HEIGHT);

    }

    ~FastLane(){

        Element* node=head->next[0];

        while(node){

            Element* tmp=node;

            node=node->next[0];

            delete tmp;

        }

        delete head;

    }

    int generate_random_height(){

        int level=0;

        while(random_gen(rng)<CHANCE&&level<MAX_HEIGHT)
```

```

        level++;
    return level;
}

void add(const T& value){
    vector<Element*> update(MAX_HEIGHT+1,NULL);

    Element* current=head;
    for(int i=current_height;i>=0;i--){
        while(current->next[i]&&current->next[i]->data<value)
            current=current->next[i];
        update[i]=current;
    }
    current=current->next[0];
    if(!current||current->data!=value){
        int new_height=generate_random_height();
        if(new_height>current_height){
            for(int i=current_height+1;i<=new_height;i++){
                update[i]=head;
                current_height=new_height;
            }
            Element* new_element=new Element(value,new_height);
            for(int i=0;i<=new_height;i++){
                new_element->next[i]=update[i]->next[i];
                update[i]->next[i]=new_element;
            }
        }
    }
}

bool find(const T& value)const{
    Element* current=head;
    for(int i=current_height;i>=0;i--){
        while(current->next[i]&&current->next[i]->data<value)
            current=current->next[i];
    }
    current=current->next[0];
    return current&&current->data==value;
}

void erase(const T& value){

```

```

vector<Element*> update(MAX_HEIGHT+1,NULL);

Element* current=head;
for(int i=current_height;i>=0;i--){
    while(current->next[i]&&current->next[i]->data<value)
        current=current->next[i];
    update[i]=current;
}
current=current->next[0];
if(current&&current->data==value){
    for(int i=0;i<=current_height;i++){
        if(update[i]->next[i]!=current)break;
        update[i]->next[i]=current->next[i];
    }
    delete current;
    while(current_height>0&&!head->next[current_height])
        current_height--;
}
}

void show()const{
    for(int i=0;i<=current_height;i++){
        Element* node=head->next[i];
        cout<<"Level "<<i<<": ";
        while(node){
            cout<<node->data<<" ";
            node=node->next[i];
        }
        cout<<"\n";
    }
}

};

int main(){
    FastLane<int> fastLane;
    fastLane.add(4);
    fastLane.add(8);
    fastLane.add(15);
    fastLane.add(16);

```

```

fastLane.add(23);
fastLane.add(42);
cout<<"After adding elements:\n";
fastLane.show();
cout<<endl;
cout<<"Searching 15: "<<(fastLane.find(15)?"Found":"Not Found");
cout<<endl;
cout<<"Searching 25: "<<(fastLane.find(25)?"Found":"Not Found");
fastLane.erase(8);
cout<<endl<<endl;
cout<<"After removing 8:\n";
fastLane.show();
return 0;
}

```

Output:

```

D:\versity's documents\4th s  x  +  v
After adding elements:
Level 0: 4 8 15 16 23 42
Level 1: 15 16

Searching 15: Found
Searching 25: Not Found

After removing 8:
Level 0: 4 15 16 23 42
Level 1: 15 16

Process returned 0 (0x0)   execution time : 0.139 s
Press any key to continue.

```

3. Implementation of Insertion operations on Splay tree.

Code:

```

#include<bits/stdc++.h>
using namespace std;

class TreeNode

```

```

{
public:
    int value;
    TreeNode *leftChild,*rightChild;
};

TreeNode* createNode(int value)
{
    TreeNode* Node=new TreeNode();
    Node->value=value;
    Node->leftChild=Node->rightChild=NULL;
    return Node;
}

TreeNode* rotateRight(TreeNode *currentNode)
{
    TreeNode *leftNode=currentNode->leftChild;
    currentNode->leftChild=leftNode->rightChild;
    leftNode->rightChild=currentNode;
    return leftNode;
}

TreeNode* rotateLeft(TreeNode *currentNode)
{
    TreeNode *rightNode=currentNode->rightChild;
    currentNode->rightChild=rightNode->leftChild;
    rightNode->leftChild=currentNode;
    return rightNode;
}

TreeNode* splay(TreeNode *rootNode,int value)
{
    if(rootNode==NULL||rootNode->value==value)return rootNode;
    if(rootNode->value>value)
    {
        if(rootNode->leftChild==NULL)return rootNode;
        if(rootNode->leftChild->value>value)
        {

```

```

        rootNode->leftChild->leftChild=splay(rootNode->leftChild->leftChild,value);
        rootNode=rotateRight(rootNode);
    }
    else if(rootNode->leftChild->value<value)
    {
        rootNode->leftChild->rightChild=splay(rootNode->leftChild->rightChild,value);
        if(rootNode->leftChild->rightChild!=NULL)rootNode->leftChild=rotateLeft(rootNode->leftChild);
    }
    return(rootNode->leftChild==NULL)?rootNode:rotateRight(rootNode);
}
else
{
    if(rootNode->rightChild==NULL)return rootNode;
    if(rootNode->rightChild->value>value)
    {
        rootNode->rightChild->leftChild=splay(rootNode->rightChild->leftChild,value);
        if(rootNode->rightChild->leftChild!=NULL)rootNode->rightChild=rotateRight(rootNode->rightChild);
    }
    else if(rootNode->rightChild->value<value)
    {
        rootNode->rightChild->rightChild=splay(rootNode->rightChild->rightChild,value);
        rootNode=rotateLeft(rootNode);
    }
    return(rootNode->rightChild==NULL)?rootNode:rotateLeft(rootNode);
}
}

TreeNode* insertNode(TreeNode *rootNode,int newValue)
{
    if(rootNode==NULL)return createNode(newValue);
    rootNode=splay(rootNode,newValue);
    if(rootNode->value==newValue)return rootNode;
    TreeNode *newNode=createNode(newValue);
    if(rootNode->value>newValue)
    {

```

```

        newNode->rightChild=rootNode;
        newNode->leftChild=rootNode->leftChild;
        rootNode->leftChild=NULL;
    }
    else
    {
        newNode->leftChild=rootNode;
        newNode->rightChild=rootNode->rightChild;
        rootNode->rightChild=NULL;
    }
    return newNode;
}

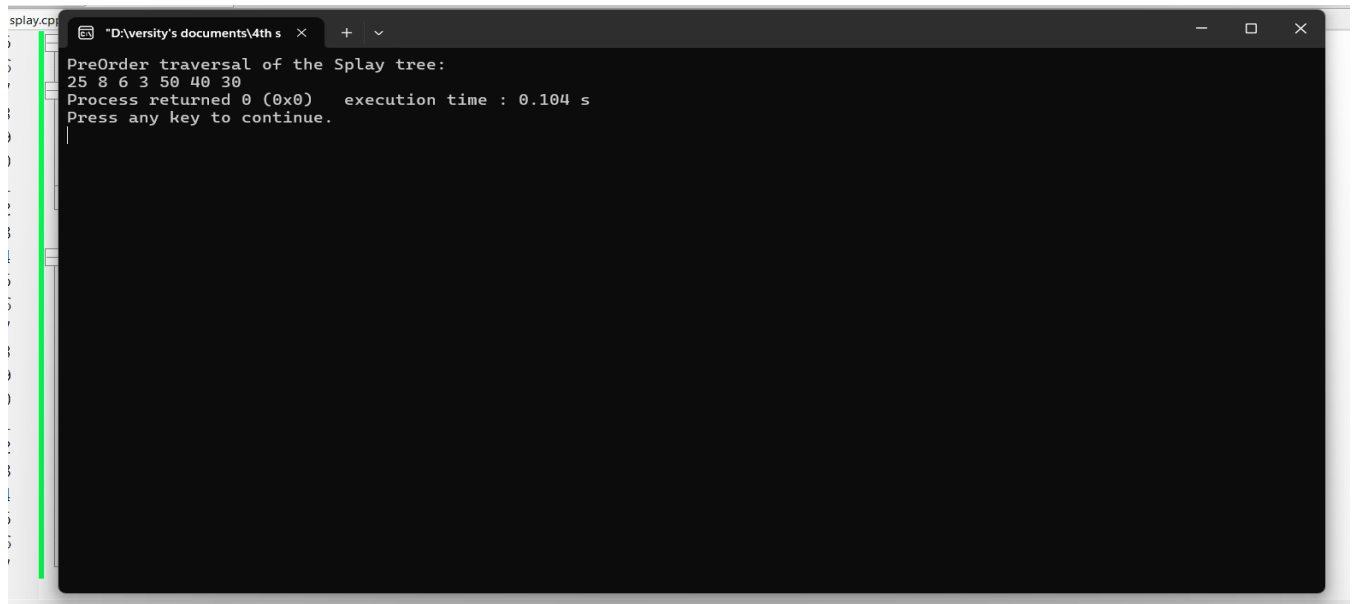
void preOrderTraversal(TreeNode *rootNode)
{
    if(rootNode!=NULL)
    {
        cout<<rootNode->value<<" ";
        preOrderTraversal(rootNode->leftChild);
        preOrderTraversal(rootNode->rightChild);
    }
}

int main()
{
    TreeNode *rootNode=createNode(6);
    rootNode->leftChild=createNode(3);
    rootNode->rightChild=createNode(8);
    rootNode->leftChild->leftChild=createNode(50);
    rootNode->leftChild->leftChild->leftChild=createNode(40);
    rootNode->leftChild->leftChild->leftChild->leftChild=createNode(30);
    rootNode=insertNode(rootNode,25);
    cout<<"PreOrder traversal of the Splay tree:";
    cout<<endl;
    preOrderTraversal(rootNode);
    return 0;
}

```

```
}
```

Output:



```
splay.cpp
D:\versity's documents\4th s
PreOrder traversal of the Splay tree:
25 8 6 3 50 40 30
Process returned 0 (0x0)   execution time : 0.104 s
Press any key to continue.
```

4. Write a program to implement a Binomial Heap and perform the following operations on the keys [10, 20, 5, 15, 30]:

- Insert all keys.
- Extract the minimum key.

Code:

```
#include<bits/stdc++.h>
using namespace std;
class N
{
public:
    int k;
    int d;
    N*p;
```



```
N*c;
```

```
N*s;
```

```
N(int x):k(x),d(0),p(NULL),c(NULL),s(NULL) {}  
};
```

```
class BH
```

```
{
```

```
private:
```

```
N*h;
```

```
static N*m(N*a,N*b)
```

```
{
```

```
N*t=new N(0);
```

```
N*y=t;
```

```
while(a!=NULL&&b!=NULL)
```

```
{
```

```
if(a->d<=b->d)
```

```
{
```

```
y->s=a;
```

```
a=a->s;
```

```
}
```

```
else
```

```
{
```

```
y->s=b;
```

```
b=b->s;
```

```
}
```

```
y=y->s;
```

```
}
```

```
y->s=(a!=NULL)?a:b;
```

```

    N*r=t->s;

    delete t;

    return r;
}

```

```

void l(N*p,N*c)
{
    c->p=p;
    c->s=p->c;
    p->c=c;
    p->d++;
}

```

```

void c()
{
    if(h==NULL)return;

```

```

    N*pr=NULL;
    N*cu=h;
    N*ne=cu->s;

```

```

while(ne!=NULL)
{
    if(cu->d!=ne->d||(ne->s!=NULL&&ne->s->d==cu->d))
    {
        pr=cu;
        cu=ne;
    }
    else
    {
        if(cu->k<=ne->k)
        {

```

```

        cu->s=ne->s;
        l(cu,ne);
    }
    else
    {
        if(pr==NULL)
        {
            h=ne;
        }
        else
        {
            pr->s=ne;
        }
        l(ne,cu);
        cu=ne;
    }
}
ne=cu->s;
}
}

```

public:

```

BH():h(NULL) {}

```

```

~BH()

```

```

{
    while(h!=NULL)
    {
        e();
    }
}

```

```
void i(int x)
```

```
{
```

```
    BH t;
```

```
    t.h=new N(x);
```

```
    u(t);
```

```
}
```

```
void u(BH&t)
```

```
{
```

```
    N*r=m(h,t.h);
```

```
    h=r;
```

```
    t.h=NULL;
```

```
    c();
```

```
}
```

```
int e()
```

```
{
```

```
    if(h==NULL)
```

```
    {
```

```
        throw runtime_error("Empty");
```

```
    }
```

```
    N*mn=h;
```

```
    N*pm=NULL;
```

```
    N*cu=h;
```

```
    N*pr=NULL;
```

```
    while(cu!=NULL)
```

```
    {
```

```
        if(cu->k<mn->k)
```

```
        {
```

```

        mn=cu;

        pm=pr;
    }

    pr=cu;

    cu=cu->s;
}

if(pm!=NULL)
{
    pm->s=mn->s;
}
else
{
    h=mn->s;
}

BH ch;
N*chd=mn->c;
N*pch=NULL;
while(chd!=NULL)
{
    N*nch=chd->s;
    chd->s=pch;
    chd->p=NULL;
    pch=chd;
    chd=nch;
}
ch.h=pch;

u(ch);

int mk=mn->k;

```

```

        delete mn;
        return mk;
    }
};

int main()
{
    BH bh;
    int a[] = {10,20,5,15,30};
    for(int x:a)
    {
        bh.i(x);
    }

    cout<<"Extracted Minimum Key: "<<bh.e()<<endl;

    return 0;
}

```

Output:

The screenshot shows a Windows command prompt window titled "Binomial heap.cpp". The output of the program is displayed as follows:

```

Extracted Minimum Key: 5
Process returned 0 (0x0)   execution time : 0.198 s
Press any key to continue.

```

The background of the command prompt is black, and the text is white. The window has standard Windows window controls (minimize, maximize, close) in the top right corner. The taskbar at the bottom shows several open applications, including a search bar, file explorer, and various development tools.

5. keys = [10, 22, 37, 40, 52, 64, 77, 88, 99, 110]. Create a hash table instance with a size of 10. Insert each key into the hash table. Print the contents of the hash table, indicating any collisions that occurred and how to resolve them.

Code:

```
#include <bits/stdc++.h>
using namespace std;

class X {
private:
    int y;
    vector<list<int>>>z;

    int a(int b) {
        return b%y;
    }
public:
    X(int c):y(c) {
        z.resize(y);
    }

    void p(int d) {
        int e=a(d);
        z[e].push_back(d);
    }

    void q() {
        for (int f=0;f<y;++f) {
            cout << "T " <<f<< ": ";

            if (z[f].empty()) {
                cout<<"EMPTY";
            }
        }
    }
};
```

```

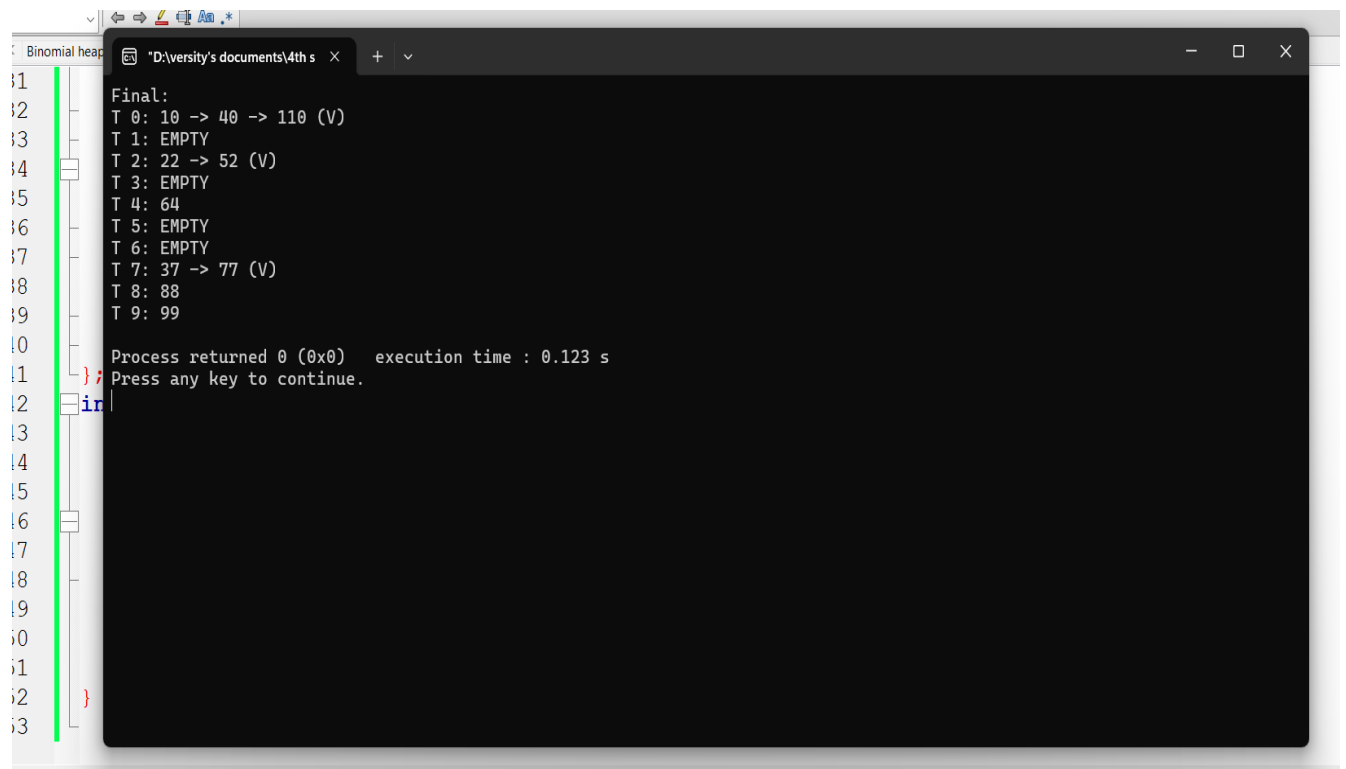
else {
    for (auto g=z[f].begin();g!=z[f].end();++g) {
        cout << *g;
        if (next(g)!=z[f].end()) {
            cout<<" -> ";
        }
    }
    if (z[f].size(>1) {
        cout<<" (V)";
    }
}
cout<<endl;
}
};

int main() {
    vector<int>h={10,22,37,40,52,64,77,88,99,110};
    X i(10);

    for (int j:h) {
        i.p(j);
    }
    cout<<"Final:"<<endl;
    i.q();
    return 0;
}

```


Output:



The screenshot shows a terminal window with a dark background and white text. The window title is "D:\versity's documents\4th s". The output of the program is as follows:

```
Final:
T 0: 10 -> 40 -> 110 (V)
T 1: EMPTY
T 2: 22 -> 52 (V)
T 3: EMPTY
T 4: 64
T 5: EMPTY
T 6: EMPTY
T 7: 37 -> 77 (V)
T 8: 88
T 9: 99

Process returned 0 (0x0)   execution time : 0.123 s
}; Press any key to continue.
```

Below the terminal window, a portion of the C++ source code is visible, showing a loop structure with a variable `ir`.

End